

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: INTERNET PROGRAMMING
COURSE CODE: CSA4305

COURSE OUTCOME COVERED

CO5: Construct interoperable web services by leveraging JAX-RPC, WSDL, SOAP, and XML Schema for seamless data exchange across distributed systems.(L4,L5)

Assessment Tool : Alternative Solution Design
Weightage: 20%

QUESTIONS:

Total Marks: 25

1. A company needs to develop a web service for a mobile and web application. Analyze both REST and SOAP approaches and design an appropriate solution by selecting one. Justify your choice based on performance, scalability, and security requirements.

ANSWER:

REST vs SOAP – Selection and Design of a Web Service **Introduction**

A company developing a **mobile and web application** needs a communication mechanism between the client applications and the backend server. Two commonly used approaches are **REST (Representational State Transfer)** and **SOAP (Simple Object Access Protocol)**.

Both can provide web services, but they differ in architecture, message format, performance, flexibility, and security.

Comparison of REST and SOAP

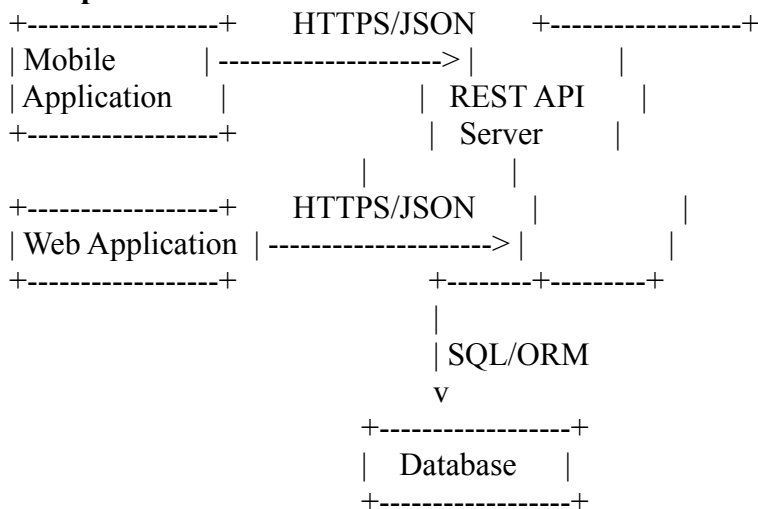
Feature	REST	SOAP
Architecture	Architectural style	Protocol
Data format	Mainly JSON; XML also possible	XML
Performance	Generally faster and lightweight	More overhead due to XML/envelopes
Scalability	Highly scalable and stateless	Scalable, but generally heavier
Mobile applications	Very suitable	Less suitable for lightweight mobile APIs
Security	HTTPS, OAuth 2.0, JWT, etc.	HTTPS + WS-Security and other WS-* standards
Flexibility	High	More rigid and contract-oriented
Bandwidth	Lower	Higher
Error handling	HTTP status codes	SOAP Fault

Best suited for Mobile, web, cloud APIs Enterprise systems requiring strict contracts

Selected Approach: REST

For a modern mobile and web application, **REST API** is the appropriate choice.

Proposed REST Architecture



Example Request

GET /api/users/1001
Authorization: Bearer <token>
Accept: application/json

Example Response

```
{
  "id": 1001,
  "name": "Ravi",
  "email": "ravi@example.com"
}
```

Why REST is Selected

1. Performance

REST commonly uses JSON, which is compact and easier to process than SOAP's XML envelope. This reduces network bandwidth and processing overhead.

2. Scalability

REST APIs are generally **stateless**, meaning each request contains the information necessary for processing it. This makes it easier to distribute requests across multiple servers.

3. Mobile Compatibility

Mobile applications benefit from lightweight JSON responses and HTTP-based communication.

4. Security

REST can use:

- HTTPS/TLS
- OAuth 2.0
- JWT
- API keys where appropriate
- Role-based authorization

Rate limiting

5. Easy Integration

REST APIs are widely supported by browsers, mobile platforms, programming languages, and cloud platforms.

Conclusion

For a general-purpose **mobile and web application**, REST provides a good combination of **performance, scalability, simplicity, and security**. SOAP would be preferred instead if the application had strict enterprise requirements involving WS-Security, WS-ReliableMessaging, or other WS-* standards.

2. An online platform expects a large number of users accessing its services simultaneously. Design a scalable service architecture showing how client, server, API, and database interact. Propose an alternative design to improve scalability and justify your approach.

ANSWER:

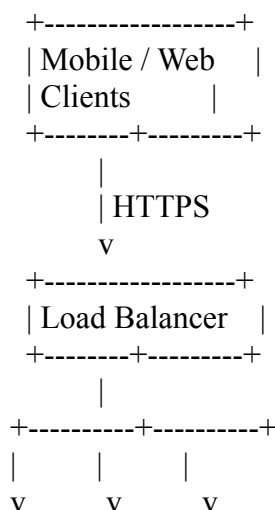
Scalable Service Architecture Introduction

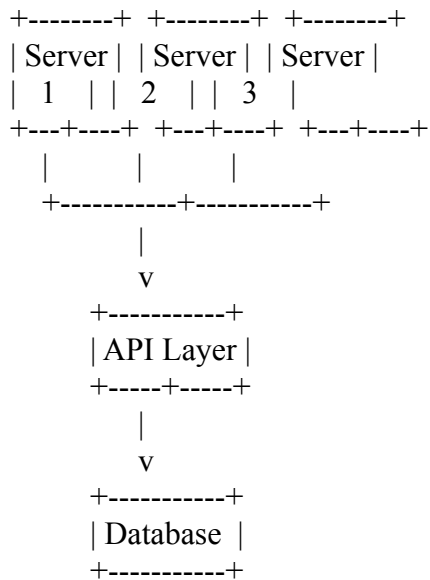
An online platform may receive thousands or millions of simultaneous requests. A simple architecture with one server can become a bottleneck.

Therefore, the architecture should support:

- Horizontal scaling
- Load balancing
- Caching
- Database scalability
- Fault tolerance
- High availability

Basic Architecture





Working

1. Client

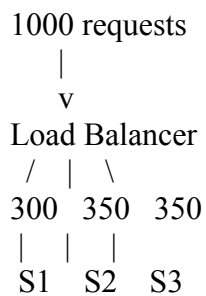
The user accesses the platform using:

- Web browser
- Android application
- iOS application

2. Load Balancer

The load balancer receives incoming requests and distributes them among available servers.

For example:



This prevents one server from becoming overloaded.

3. Application Servers

Multiple application servers process requests.

If traffic increases, more servers can be added.

4. API Layer

The API layer provides endpoints such as:

- GET /products
- POST /orders

GET /users/1001
PUT /users/1001

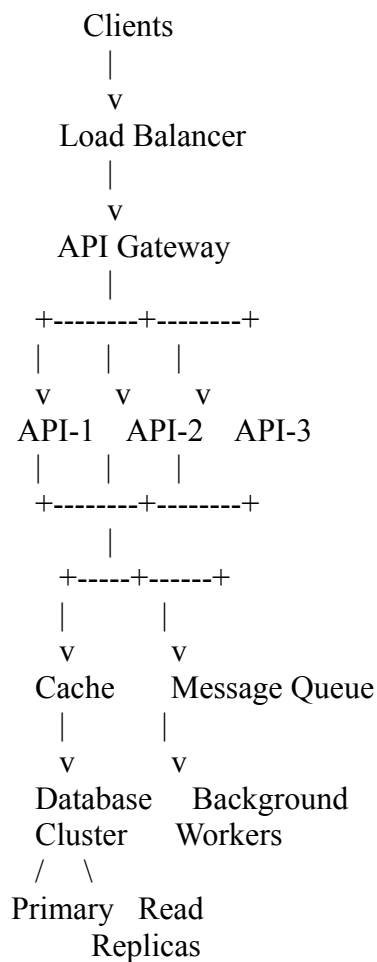
5. Database

The database stores application information.

However, a single database can become a bottleneck.

Alternative Design: Caching + Database Read Replicas

A better scalable architecture can introduce **distributed caching and database replication**.



Why This Alternative is Better

Distributed Cache

Frequently requested data can be stored in a cache such as Redis.

Instead of:

Client → API → Database

the system can use:

Client → API → Cache

Cache Hit
|

Response

This reduces database load.

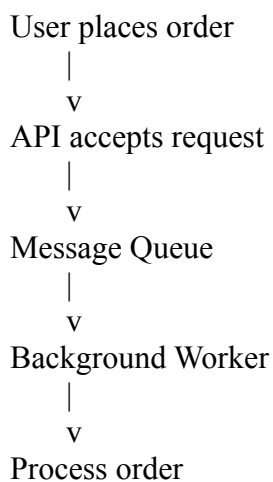
Database Read Replicas

Read-heavy applications can send read requests to replicas while writes go to the primary database.

Message Queue

Long-running operations can be processed asynchronously.

For example:



This prevents slow operations from blocking users.

Benefits

The alternative architecture provides:

- Higher throughput
- Better fault tolerance
- Lower database load
- Faster response times
- Horizontal scalability
- Better availability

3. A banking application requires secure communication between client and server. Analyze available communication protocols and select the most suitable protocol. Justify your selection based on security, reliability, and performance.

ANSWER:

Secure Communication Protocol for Banking Application Introduction

A banking application handles highly sensitive information such as:

- Account details
- Passwords
- Transaction information
- Personal information
- Authentication credentials

Therefore, communication between client and server must provide **confidentiality, integrity, authentication, and reliability**.

Available Protocols

HTTP

HTTP does not encrypt application traffic.

Therefore, plain HTTP is **not suitable for banking transactions**.

HTTPS

HTTPS is HTTP protected by **TLS (Transport Layer Security)**.

It provides:

- Encryption
- Server authentication
- Integrity protection

TCP

TCP provides reliable transport and ordered delivery, but **TCP itself does not encrypt application data**.

UDP

UDP is faster and connectionless, but it does not provide TCP-style reliable delivery. It is generally not appropriate as the primary transport for ordinary banking web transactions.

WebSocket over TLS

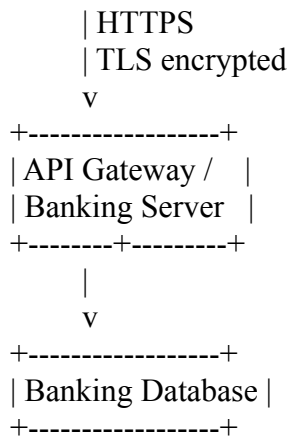
Secure WebSockets (wss) can be useful for real-time communication, such as live transaction notifications, but are not necessary for every banking API.

Selected Protocol: HTTPS with TLS

The most suitable choice for standard banking client-server communication is:

HTTPS over TLS

```
+-----+
| Banking Client |
+-----+-----+
|
```



Why HTTPS/TLS is Selected

1. Confidentiality

TLS encrypts information during transmission.

An attacker intercepting the network traffic should not be able to read the protected contents.

2. Authentication

The server presents a digital certificate so the client can authenticate the server.

3. Integrity

TLS helps detect modification of protected data during transmission.

4. Reliability

HTTPS commonly runs over TCP, which provides ordered and reliable delivery for traditional HTTP communication.

5. Performance

Modern TLS versions such as TLS 1.3 reduce connection and handshake overhead compared with older approaches while providing strong security.

Additional Banking Security

HTTPS alone is not sufficient for the entire banking security model. The application should also implement:

- Multi-factor authentication
- Strong password policies
- Session management
- Authorization
- Transaction signing/confirmation where appropriate
- Rate limiting
- Fraud detection
- Secure logging
- Encryption of sensitive data at rest

Conclusion

HTTPS with modern TLS is the most suitable standard protocol for secure banking client-server communication because it provides a strong combination of **security, reliable transport, and acceptable performance**.

4. **An application needs to integrate with third-party services such as payment gateways or maps. Design an API integration strategy, including request handling, authentication, and response processing. Suggest an alternative approach to improve efficiency or reliability.**

ANSWER:

API Integration Strategy for Third-Party Services

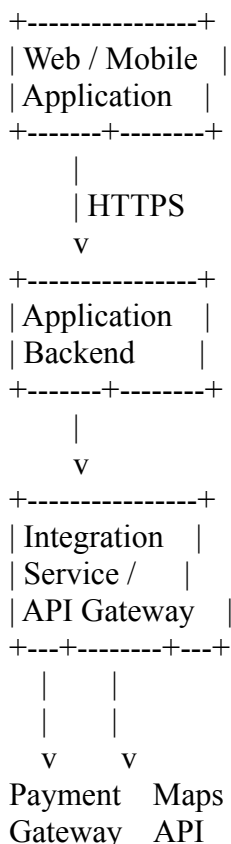
Introduction

An application may need to communicate with external services such as:

- Payment gateways
- Google Maps or other mapping services
- SMS services
- Email services
- Cloud storage
- Authentication providers

The application should use a well-designed API integration strategy rather than directly scattering third-party API calls throughout the application.

Proposed Architecture



Step 1: Request Handling

The client sends a request to the application's own backend.

For example:

```
POST /api/payment
Content-Type: application/json
Authorization: Bearer <token>
{
  "amount": 1000,
  "currency": "INR",
  "orderId": "ORD1001"
}
```

The backend validates the request.

Step 2: Authentication

The backend authenticates the user and then authenticates with the third-party service using the method required by that provider.

Possible mechanisms include:

- API keys
- OAuth 2.0 access tokens
- Signed requests
- Mutual TLS in specialized enterprise environments

Secret API keys should be stored on the server or in a secrets-management system, not inside a mobile application.

Step 3: Third-Party API Request

The integration layer sends a request to the external service.

```
Application
|
v
Integration Service
|
Authenticated HTTPS request
v
Third-Party API
```

Step 4: Response Processing

The third-party service returns a response.

Example:

```
{
```

```
"status": "success",  
"transactionId": "TXN5001"  
}
```

The integration service validates the response and converts it into the application's internal format.

Step 5: Return Response to Client

The backend sends a clean response to the application:

```
{  
  "paymentStatus": "SUCCESS",  
  "transactionId": "TXN5001"  
}
```

Error Handling in API Integration

Third-party services can fail because of:

- Network timeout
- Invalid credentials
- Rate limits
- Service downtime
- Invalid request
- Server errors

The application should therefore implement:

Timeout

Do not wait indefinitely for an external API.

Retry

Retry temporary failures using controlled **exponential backoff** and a maximum retry limit.

Idempotency

For operations such as payments, use an idempotency key where supported so that a retry does not accidentally create duplicate transactions.

Circuit Breaker

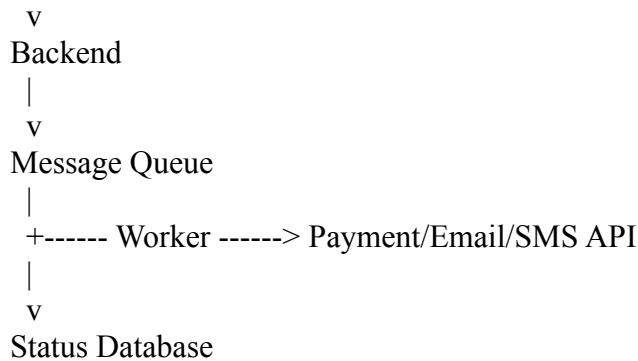
If a third-party service is continuously failing, temporarily stop sending requests to it and return a controlled response.

Alternative Approach: Asynchronous Integration

For non-immediate operations, a **message queue** can improve reliability.

Client

|



For example, sending an email does not always need to happen before the user's HTTP request finishes.

Advantages

- Reduces user waiting time
- Handles temporary third-party failures
- Supports retry
- Prevents traffic spikes from overwhelming the external API
- Improves overall reliability

For operations that require an immediate result, such as a payment authorization response, synchronous processing may still be necessary.

5. A web service frequently encounters errors during client requests. Design an error handling and fault tolerance mechanism. Propose alternative strategies to improve system reliability and ensure smooth user experience.

ANSWER:

Error Handling and Fault Tolerance Mechanism Introduction

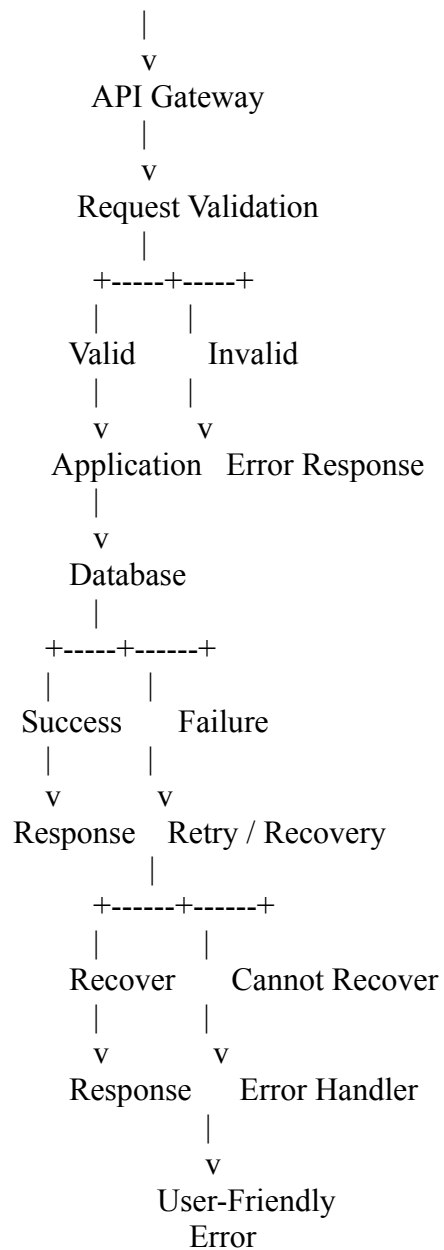
Web services can fail because of:

- Invalid client requests
- Authentication failures
- Server errors
- Database failures
- Network problems
- External API failures
- High traffic
- Hardware failures

Therefore, the system needs a structured **error-handling and fault-tolerance mechanism**.

Proposed Error Handling Architecture

Client



Types of Errors

1. Client Errors

These occur because of incorrect client requests.

Examples:

- Missing parameter
- Invalid JSON
- Invalid authentication
- Invalid resource ID

Use appropriate HTTP status codes such as:

- 400 Bad Request
- 401 Unauthorized
- 403 Forbidden
- 404 Not Found
- 409 Conflict
- 422 Unprocessable Content

2. Server Errors

These occur because of problems on the server.

Examples:

- Database failure
- Programming error
- Service unavailable

Common responses include:

- 500 Internal Server Error
- 502 Bad Gateway
- 503 Service Unavailable
- 504 Gateway Timeout

The client should receive a safe, meaningful error message rather than internal stack traces.

Centralized Error Handler

Instead of implementing different error responses throughout the application, use a centralized error-handling mechanism.

Example:

Application Error

|

v

Central Error Handler

|

+--> Log technical details

|

+--> Generate error ID

|

+--> Select HTTP status

|

+--> Return safe response

Example response:

```
{
  "error": {
    "code": "SERVICE_UNAVAILABLE",
    "message": "The service is temporarily unavailable.",
    "requestId": "REQ10025"
  }
}
```

The user receives a clear message, while detailed technical information remains in secure logs.

Fault Tolerance Techniques

1. Retry Mechanism

Temporary failures can be retried.

Example:

Request

|

v

Failure

|

v

Retry 1

|

v

Failure

|

v

Retry 2

|

v

Success

Use exponential backoff to avoid overwhelming a failing service.

2. Circuit Breaker

A circuit breaker protects the system from repeatedly calling a failed dependency.

Normal

|

v

CLOSED

|

Repeated failures

|

v

OPEN

|

Wait

|

v

HALF-OPEN

|

Test request

/\

OK Fail

|

|

v

v

CLOSED OPEN

Benefits

- Prevents cascading failures
- Reduces unnecessary requests
- Allows failed services time to recover

3. Timeout

Every external call should have a reasonable timeout.

Instead of:

Client → Server → Waiting forever

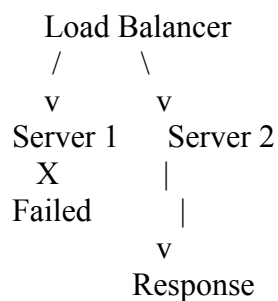
use:

Client → Server → Timeout
|
v
Controlled error

4. Load Balancing

Multiple servers can handle requests.

If one server fails:



The load balancer can route traffic to healthy servers.

5. Caching

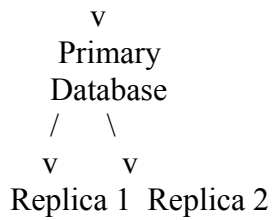
Frequently requested data can be cached to reduce:

- Database load
- Network calls
- Response time

6. Database Replication

A primary database can have replicas to improve availability and read scalability.

Application
|



7. Monitoring and Logging

The system should monitor:

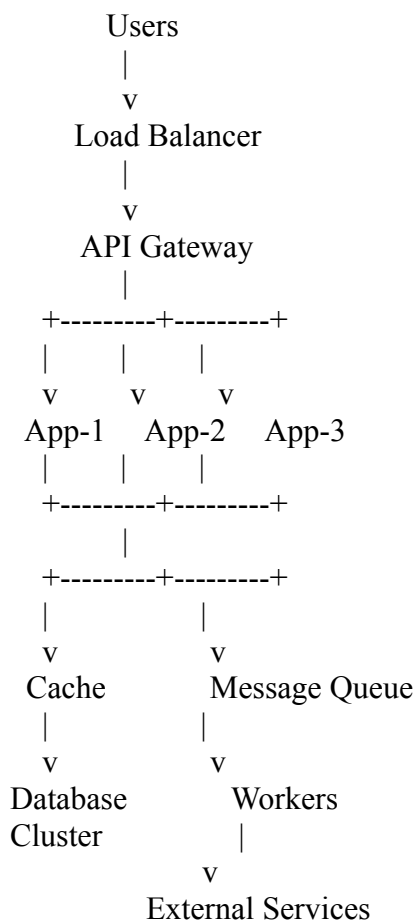
- Error rate
- Response time
- CPU and memory
- Database performance
- API availability
- Request volume

Logs should contain useful diagnostic information without unnecessarily exposing sensitive data.

Alternative Reliability Strategy

A more advanced design can combine:

API Gateway + Load Balancer + Multiple Servers + Cache + Message Queue + Database Replication + Circuit Breaker



Why This Improves Reliability

Load balancing prevents server overload.

Multiple application servers eliminate a single application-server failure point.

Caching reduces database pressure.

Message queues handle asynchronous workloads.

Retries handle temporary failures.

Circuit breakers prevent cascading failures.

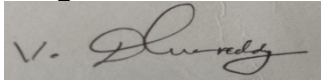
Database replication improves availability.

Monitoring detects failures early.

Code of Conduct:

I V. Dhara nireddy (1 9 2 3 1 1 2 1 9) ____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

A handwritten signature in black ink, appearing to read 'V. Dhara nireddy', is written on a light-colored rectangular background.

RUBRICS FOR CALCULATION:

Criteria	Weight age (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	20%	Clearly defines problem, objectives, and all requirements accurately	Understands problem with minor gaps	Partial understanding; misses key requirements	Misinterprets or fails to understand problem
Generation of Solutions	25%	Develops multiple innovative and feasible alternatives with clear differentiation	Generates relevant alternatives with minor limitations	Limited or repetitive alternatives	Fails to generate meaningful alternatives
Evaluation of Alternatives	25%	Critically compares alternatives using appropriate criteria and metrics	Compares alternatives with some justification	Basic or superficial comparison	No proper comparison conducted
Solution Selection	20%	Selects best solution with strong justification and decision rationale	Selects suitable solution with moderate justification	Selection is weakly justified	No clear or justified selection
Feasibility	10%	Applies relevant concepts ensuring high feasibility and practicality	Applies concepts with minor issues	Limited feasibility or incorrect application	Solution is impractical or conceptually incorrect